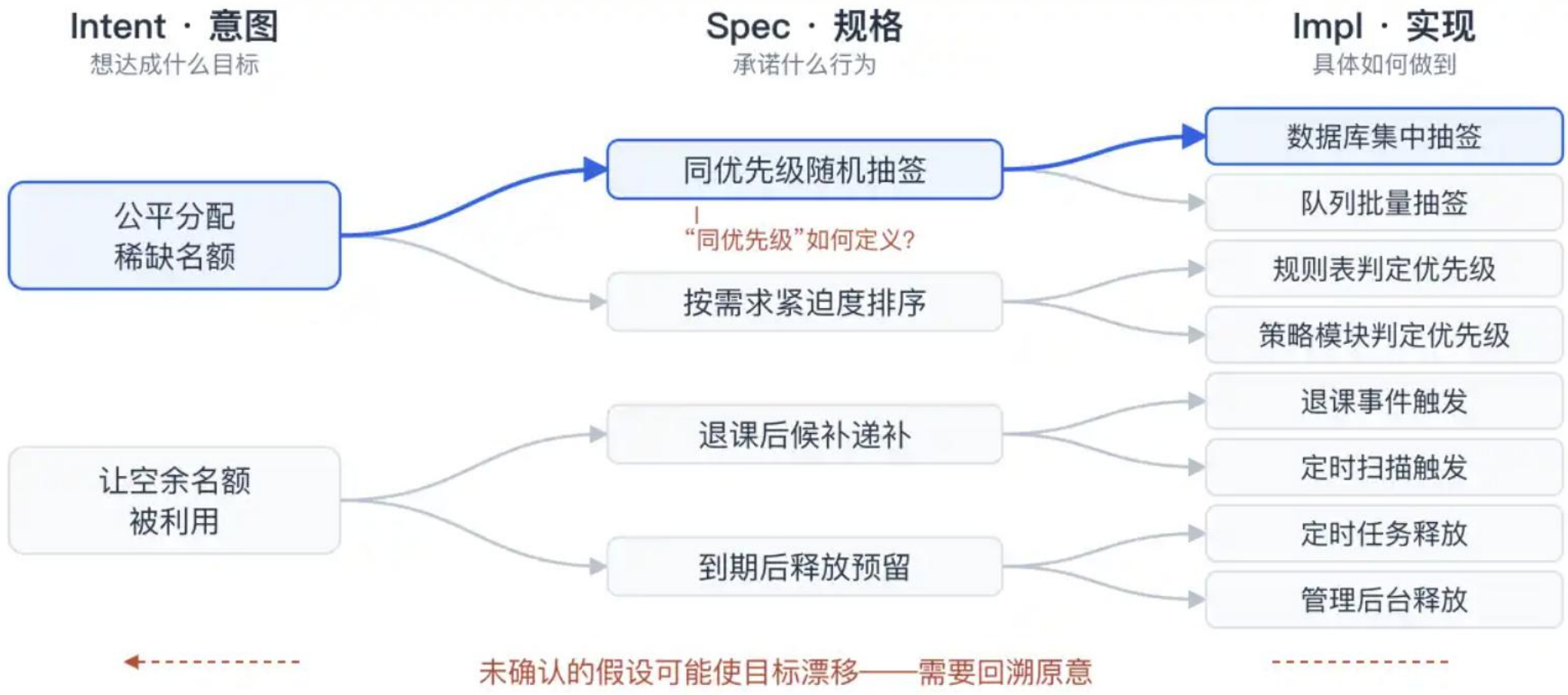
Intent → Spec → Impl

看似明确的需求，其实都有很多开放的空间



架构无处不在

> 代码之外有客户的规律；客户之外有社会的规律；社会之外有宇宙的规律

从 10 行到 500,000 行

- 10 行的架构：Tower of Hanoi
- 20,000 行的架构：QRGen
- 500,000 行的架构：教务系统（今天继续）

可行架构仍有改进的空间

- “脑子一热” 的可行架构往往还有改进的空间

软件工程的本质：驾驭复杂性

Problem Domain → Essential Complexity

- 所选问题本身要求处理的复杂性：课程容量、先修要求、时间冲突。
  - 即使换成人工办理，这些约束仍然存在。

技术实现 → Accidental Complexity

- 实现方式额外引入的复杂性：重复数据、格式转换、缓存同步、部署配置。
  - 业务规则没变，换一种实现，就可能省掉许多麻烦。
- 这一区分来自 Brooks 的 No Silver Bullet：区分问题本身的困难与表达、实现它时附带的困难。

需求工程：直面 Essential Complexity

收缩 Intent Space 的不确定性

- 需求越完备，歧义越小
  - “课程容量 100 人”：候补算不算？跨院预留名额能否借用？

但管理代价也越大

- 每个需求点都带来审计的代价
- 换到今天，是 token 开销（和模型潜在的错误发挥）
  - 每补上一条约束，也多了一条需要持续核对的承诺。

架构设计

分解 Essential Complexity；减少 Accidental Complexity

- 好的架构能更容易地实现需求、承受需求的变更
  - 把容量、资格、时间冲突分开建模，让每一部分都能单独理解。
  - 让页面、批处理共用规则接口，减少重复实现与口径不一。

收缩 Spec / Impl Space 的可能性

- 明确模块职责、数据归属和接口约定，排除大量随意组合。
- 一个实用判断：**需求改一处，需要跟着改多少处?**

On Our Inability to Do Much

- *Structured Programming* (O.-J. Dahl, E. W. Dijkstra, and C. A. R. Hoare) , 1972

> “ ...I have a very small head and I had better learn to live with it and to respect my limitations and give them full credit, rather than to try to ignore them...



Our Mental Aids

枚举、归纳、抽象

- Dijkstra 的 Part I 讨论了 our mental aids：**枚举、归纳、抽象**。
- 虽然讨论的案例在今天看来都是 “小程序”，但 necklace (pearls) 的比喻就是在分解复杂性。

Necklace (Pearls)：设计决策之间的依赖

- 每颗 pearl 承载一项独立设计决策；决定的先后，会改变其他层必须知道多少细节。
  - 过早把 image 解释成 lines，中间各层就都要理解 lines；推迟表示决策，更多层才能独立于它。原文：On What We Have Achieved

Hierarchical Program Structures

Small Head

- Part III： “precise thinking is possible only in terms of a small number of elements at a time…”
- SIMULA class/object abstraction

控制流解耦： Subroutine vs. Coroutine

- 原书的例子：让两个下棋程序互相对弈。谁应当是主程序，谁应当是子程序?
- 子程序服从 “调用—返回”；协程让双方保留各自的执行位置，交替恢复。

Part III, §4, pp. 184–185

让 AI 带我们读第七章

Part III, §7: Concept Hierarchies

- 原文 pp. 208–220：逐层缩短问题概念与机器概念之间的距离。
- 给 AI 的任务：沿 **TWLIST → MINISIM → Job Shop Model** 阅读。
  - 每层新增了什么概念，让使用者可以暂时忘掉什么细节？请用原文和页码说明。

站在今天的角度

- 在每个系统中都能看到这些想法的影子；有些成功、有些失败。
- 带着证据比较：Python 协程保留了什么？Go 不采用类型继承层次又舍弃了什么?

阅读经典 (1990s)

> One thing the human mind seems to do well is to name things; we have powerful associative memories. Given a name, we can quickly recall some associated thing to mind. (Foreword of SICP, Guy L. Steele Jr)

Guy L. Steele Jr.

- 与 Gerald Jay Sussman 共同设计了 Scheme，也是 SICP 所用语言的创造者之一。
- Common Lisp: The Language 的作者。
- The Java Language Specification 的合著者。

然后我意外地发现了 Growing a langauge (OOPSLA'98 Keynote)

Growing a Language

> “ It is good to design a thing, but it can be far better (and far harder) to design a pattern...We should now think of a language design as being a pattern for language designs, a tool for making more tools of the same kind.

让听众经历语言的 “生长”

- 演讲从单音节词开始；多音节词要先定义，或用已说明的构词规则生成。听众亲身经历：先建立词汇，才能表达复杂思想。(原文，pp. 3–4)

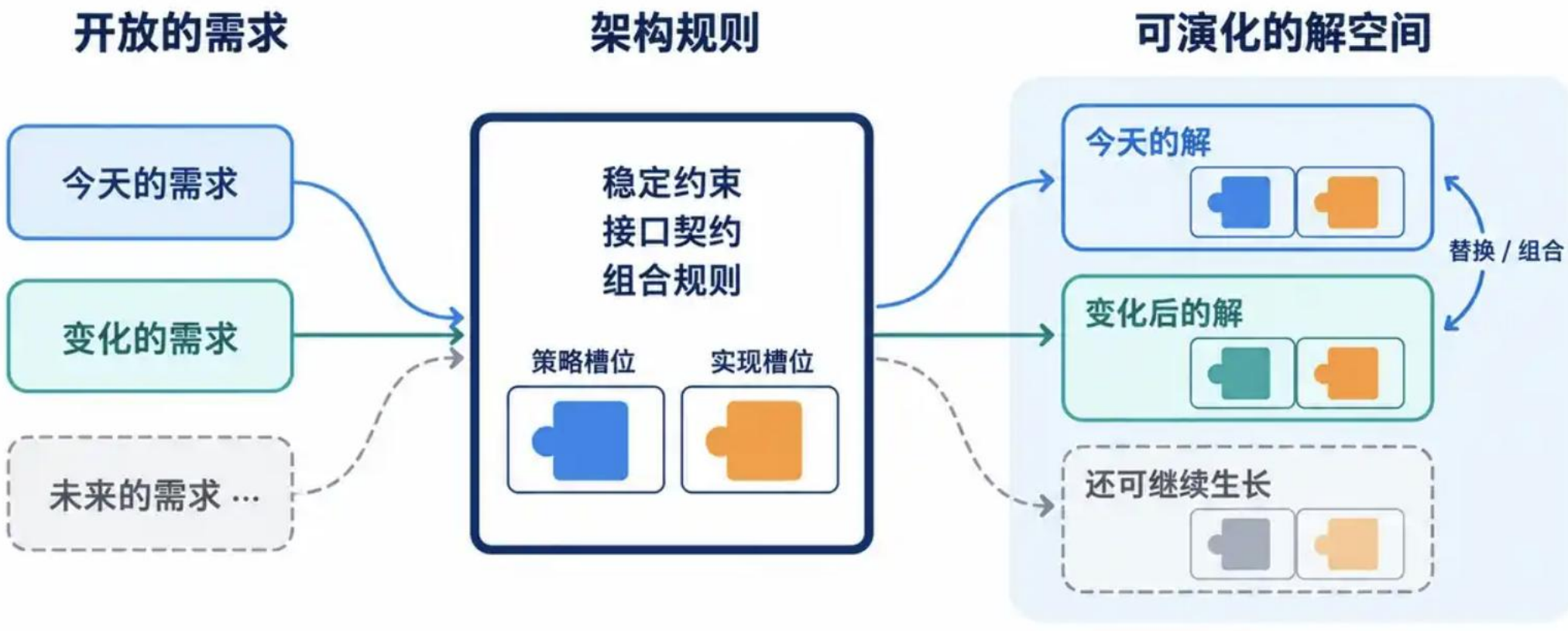
使用模式：长程任务中积累的顶级人类智慧

- 保留 slots、允许以后选择和演化的 design space
- Steele 引用建筑师 Christopher Alexander 的《俄勒冈实验》：预先定死总体规划，会夺走使用者塑造未来的机会。**哪些选择留给后来的人，也是一项设计。** (原文，pp. 8–9)

好的架构：设计解空间

固定协作规则，保留局部选择

- 好的架构不是选出一个解，而是设计**开放需求空间对应的解空间**



需求变化时，在共同规则下选择、替换、组合

阅读经典，建立品味

Comment: 有太多宝藏被 “埋在” 这个快速发展的时代了

- 海量人类产物 (CSDN、防自学教科书……)、AI slop 在疯狂污染大家的品味
- 阅读经典是成长必备的道路
- 我真的很期待一个 “大学毁灭计划”
  - 所有学校的改进空间都实在太大了

把经典读进自己的设计

- 拿自己的系统追问：作者把哪些判断交还给了使用者？这些选择，需要什么共同规则才能彼此兼容？

设计解空间：UNIX Philosophy

三条原则

- Do one thing and do it well.
- Write programs to work together.
- Write programs to handle text streams, because that is a universal interface.

精髓是第三条

- 文本接口统一了人-程序、程序-程序的接口，使用者可以
  - 自由组合程序
  - 随时检查程序的中间结果
- Mcllroy 在 1964 年就提出：像连接花园水管一样连接程序。原始备忘录

UNIX：组合需要共同约定

POSIX：命令与进程的公共接口

- 参数：短选项、选项参数、操作数；-- 结束选项解析（个别工具有历史例外）。语法约定
- 数据与诊断分开：stdin 输入、stdout 输出、stderr 诊断；| 默认连接前者的标准输出与后者的标准输入。Shell
- 退出状态单独传递：0 通常表示成功，其他值按工具约定解释；例如 grep 的 1 表示未匹配。退出状态

社区实践：让文本成为稳定的数据接口

- 简单记录常用 “一行一条”、固定字段；复杂结构用 TSV、JSON 等明确格式。
- 机器模式保持稳定；彩色、进度条、提示语不混入数据流。
- 分隔符也是协议：文件名可以包含换行，传文件列表可用 NUL 分隔。GNU 工具实践
- 排序、字符分类会受 locale 影响；需要可复现规则时可固定 LC_ALL=C。POSIX 环境变量

UNIX：使用者创造新的用途

现代工具继续扩展组合的边界

- `jq`：把 JSON 中需要的字段提取成文本流，API 响应也能进入流水线。
- `fzf`：从标准输入读取候选项，让人选择，再把结果写回标准输出。
  - 书目 JSON → `jq` 提取书名 → `fzf` 人工选择：人的判断也是可组合的一步。
- `xdotool`：把窗口搜索、键盘和鼠标操作变成命令（适用于 X11）。
  - 窗口 ID 列表 → `fzf` 选择 → `xdotool windowactivate`：组合出自己的窗口切换器。

设计者提供了组合的规则，使用者可以创造设计者没有想到的用途

案例二：关系数据库

> A relation model of data as a basis for protecting users of formatted data systems from the potentially disruptive changes in data representation caused by growth in the data bank and changes in traffic. (E. F. Codd, 1970)

把物理世界状态投射到关系数据库中

- 分离 “数据的理解” 与 “数据在机器中的组织”
- “谁借了哪本书” 保持不变，数据页、索引、访问路径可以改变。Codd 的数据独立性

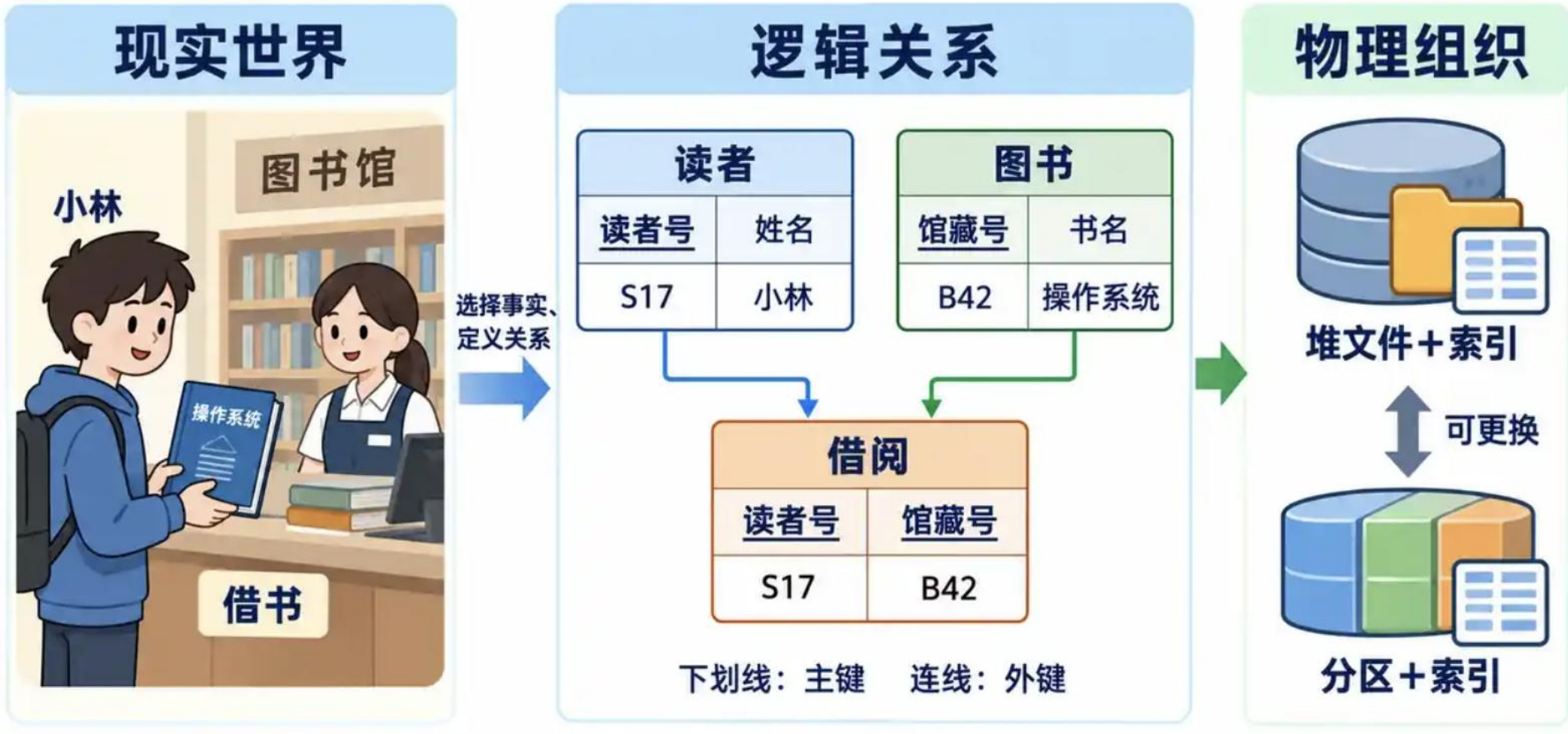
在逻辑关系上构建应用

- SQL、查询优化器、事务实现使 CRUD 成为标准的软件架构 pattern
- 随后发展的关系数据库系统，把 “声明要什么” 落实为可选择的执行计划。System R (1976)

关系数据库：现实、关系与存储

同一份数据，支持不同的问题

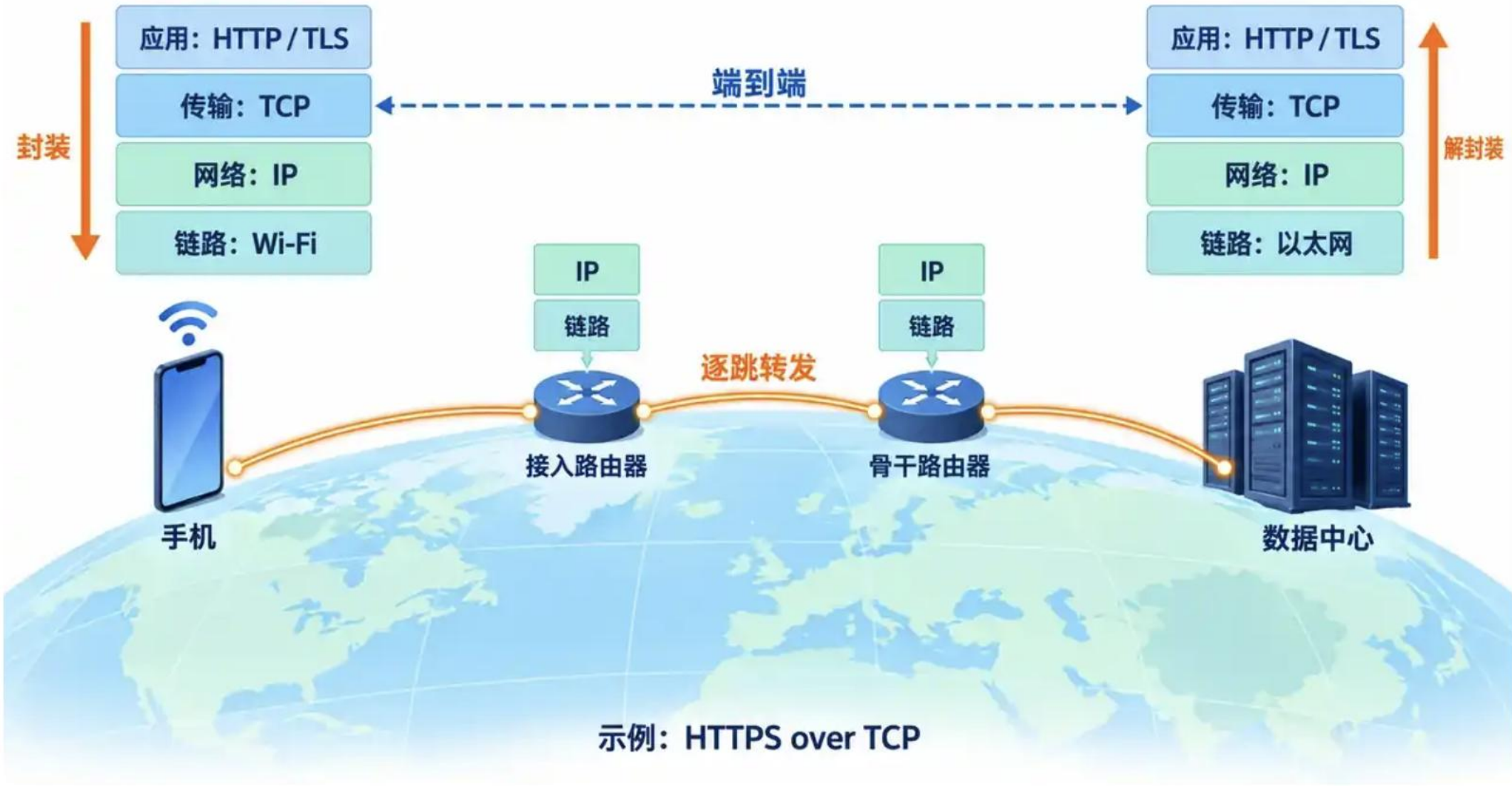
- 记录当前借阅：按读者查“借了什么”，按馆藏查“被谁借走”。



查询关心事实，存储可以改变

案例三：The Internet

手机与数据中心如何通信：TCP/IP 分层



示例：HTTPS over TCP

Internet：在持续变化中演化

RFC 1958: Architectural Principles of the Internet (1996)

> “ In searching for Internet architectural principles, we must remember that technical change is continuous in the information technology industry.

做好 “数据传输” 这一件事

- Telnet → ssh
- FTP/HTTP/... → HTTPS （许多常见用途的迁移）
- SMTP/IMAP 一直到今天还在使用

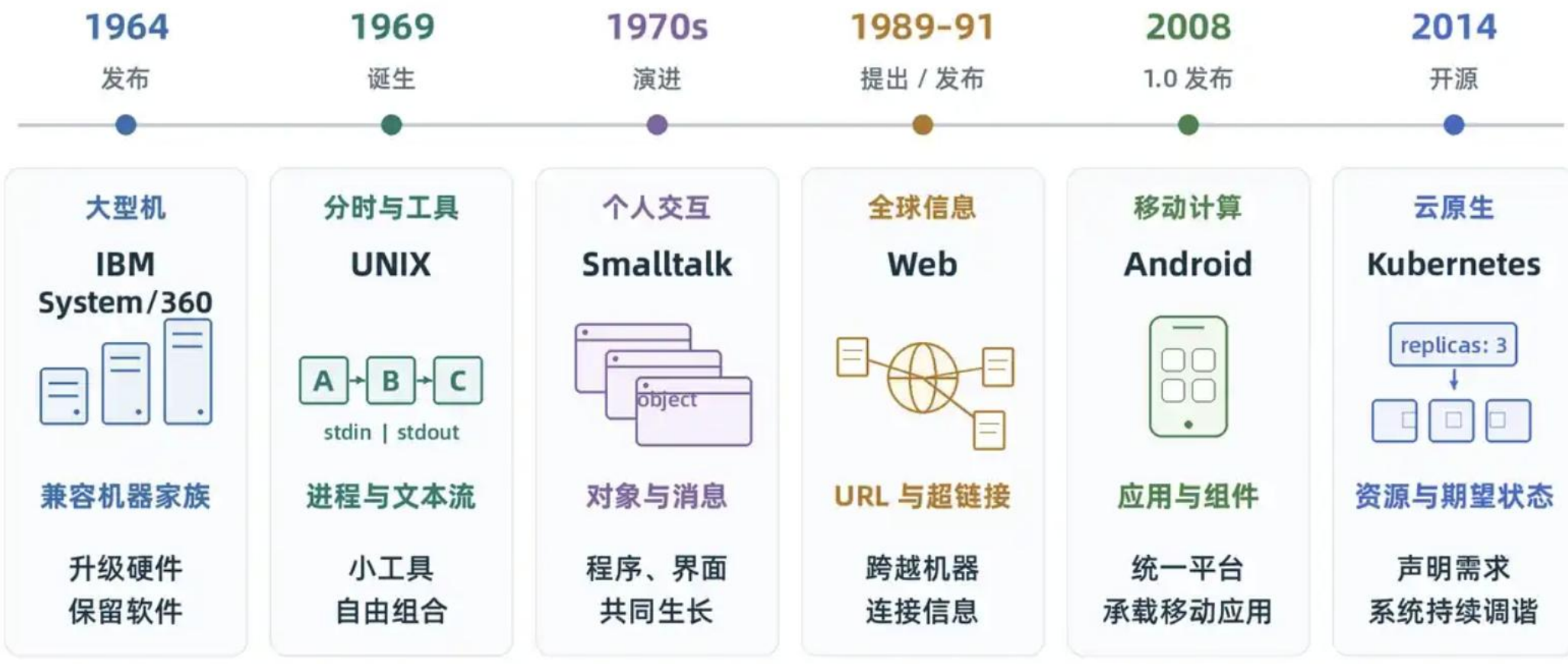
在共同规则之上，持续更换局部

- RFC 用城市作比喻：街道与建筑逐步更新，城市仍持续运转。
- 少量合作规则，可以生成庞大、多样、不断演化的技术空间。RFC 1958 §1

Systems：每一轮浪潮的新抽象

为通用应用搭建抽象层

- 刚才三个都是 “Systems”，是为了支撑通用应用做的抽象层



抽象层不断累积：让更多应用站在共同的平台上

史料：IBM · UNIX 原作者 · Alan Kay · CERN · Android · Kubernetes

如果是面向客户需求的软件呢?

操作系统、CRUD 数据库、Internet 连接了服务器和世界

- 最直接的想法：每个页面就是一个 SQL Transaction
  - 就可以支持在线商城、社交网络、教务系统……
  - 选课请求：读课程 → 检查能否选 → 写选课记录 → 返回页面

业务代码应该放在哪里?

- 学生选课页、管理员补选页，都要遵守课程容量限制
- 把规则复制到每个页面，改一次规则就得找遍所有入口

Java Web：直接在 HTML 里写代码

Java Web 浪潮 (1990s 后期)

- Java Virtual Machine: Write once, configure everywhere 😄
- 自动内存管理，程序员的救星

页面模板：让 HTML 留在 HTML 里

- PHP/FI 的前身 (1995)、ASP (1996) 已探索在页面里嵌入服务器端脚本
- Servlet：在 Java 中输出 HTML；JSP 1.0 (1999)：在 HTML 中嵌入 Java
  - JSP 由容器翻译成 Servlet：改变的是编写方式
- 页面结构清楚了，业务逻辑也很容易 “顺手” 写进页面

JSP：从 Java 片段到表达式与标签

同一张页面，两代写法

- <% ... %> / <%= ... %>：执行 Java / 输出表达式
- JSTL 1.0 (2002) 提供标准标签与 EL；JSP 2.0 (2003) 将 EL 纳入页面语言

```
<%@ page contentType="text/html;charset=UTF-8" %>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<html><body>
    <% String msg = "Hello, JSP!"; %> <!-- Scriptlet -->
    <h1><%= msg %></h1>
    <ul>
        <c:forEach var="user" items="${users}"> <!-- JSTL + EL -->
            <li>${user.name}</li>
        </c:forEach>
    </ul>
</body></html>
```

- **换成标签，不会自动得到 MVC：业务规则仍需要独立的归属。**

MVC：让不同的变化各有归属

Separation Of Concerns

- Models–Views–Controllers (1979): Separation of concerns
- 最初面向交互式界面：同一个业务对象，可以有不同的展示和操作方式

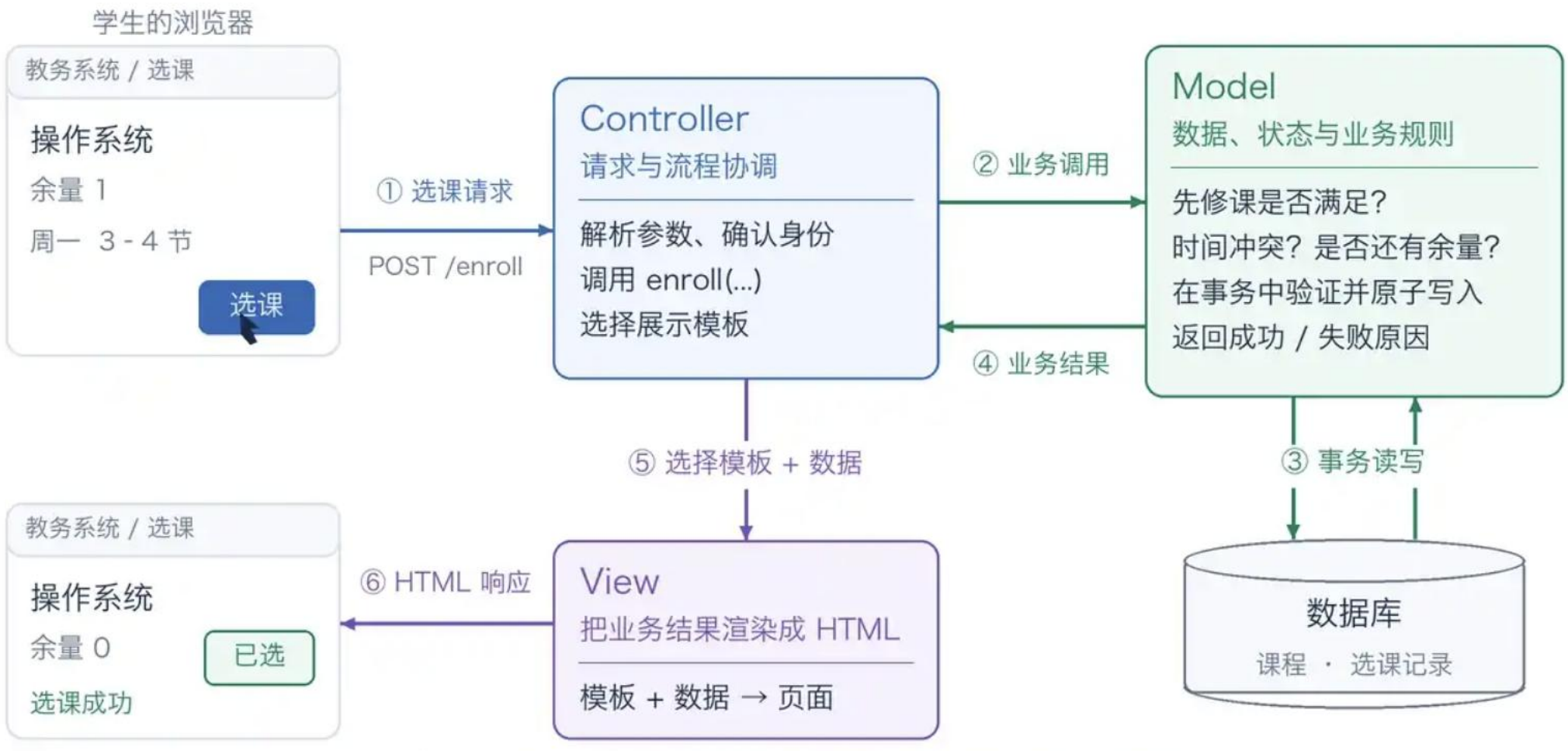
教务系统的标准架构

- Model：封装数据、状态和业务逻辑
  - 课程、选课记录、容量与冲突规则；不只是数据库里的几张表
- View：展示界面
- Controller：接收用户输入，协调 Model 和 View
- 课表从列表换成周历，主要改 View；选课限制变化，主要改 Model

MVC：一次选课请求

Web MVC 中的业务流程与界面

- 请求处理：Controller 组织请求，Model 执行业务规则，View 呈现结果



示意：现代服务器端 Web MVC；1979 年桌面 MVC 的交互机制与此不同。

把 ViewState 也当成 Model

同一份业务数据，不同的界面

- $\text{View} = \text{render}(\text{Model}, \text{ViewState})$
- 但有一个割裂的地方：ViewState 通常不会进入业务数据库
  - 展开的菜单、hover 的 Element、……
- 课程数据没变，只改筛选词 `query`，显示的列表就变了

为什么不把 ViewState 也当成一个 Model 呢?

- Presentation Model (Fowler, 2004)：将界面状态与行为从具体控件中提取出来
- **是否持久化，与是否值得建模，是两件事。**

MVVM：让界面绑定状态

用 Vue 理解 ViewModel

- Model：课程数据；View：筛选框与课程列表
- ViewModel：筛选词 `query`、派生列表 `visibleCourses` 及相关行为
- 用 `ref` 保存 `query`，用 `computed` 描述列表怎样依赖它

绑定关系写一次，变化由框架追踪

- `<input v-model="query">` 封装了两条连接：
  - 状态 → 控件：`:value="query"`
  - 输入 → 状态：`@input="query = $event.target.value"`
- 只保存 `query` 等必要状态；`visibleCourses` 从课程与筛选词派生，避免手工同步两份状态

React：用状态快照描述界面

每次 Render 都描述“此刻应该显示什么”

- `const [query, setQuery] = useState('')`：状态与更新入口
- 组件根据本轮 `query` 筛选课程、返回 JSX；派生列表不必另存为 State
- `setQuery` 请求下一次渲染，本轮拿到的状态仍是原来的快照

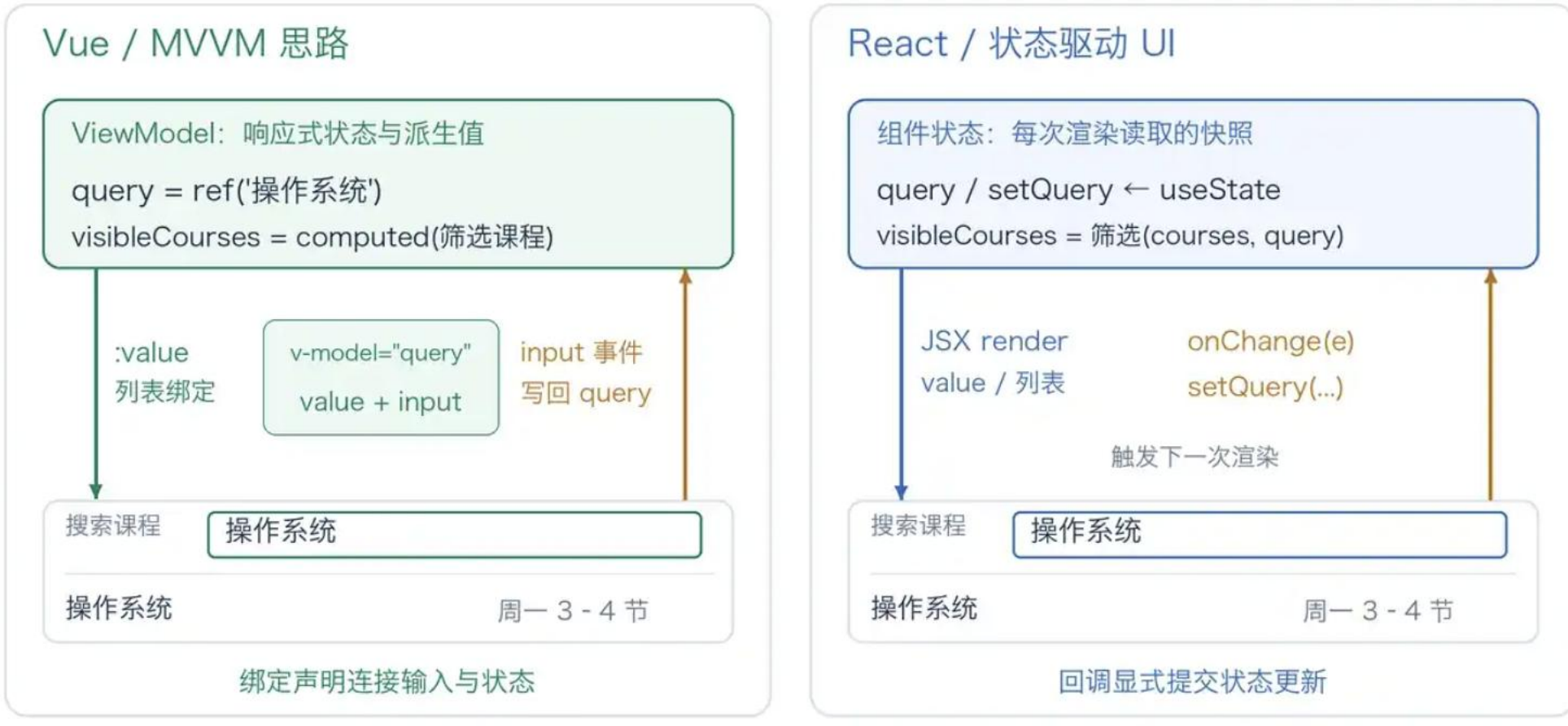
同一个输入框，显式连接事件与更新

- 状态 → 控件：`value={query}`
- 输入 → 更新：`onChange={e => setQuery(e.target.value)}`
- 受控输入框：事件 → 更新状态 → Render → 界面

Vue 与 React：同一个课程筛选框

都由状态决定界面，区别在如何表达更新

- Vue 用绑定语法连接输入与状态；React 在事件中显式调用状态更新函数



共同点：状态驱动界面，事件改变状态；Vue 组件间同样 props 向下、事件向上。

v-model 的双向连接针对绑定点；Vue 的组件 Props 与 React 的组件数据流都是从父到子。

应对复杂性

MVC/CRUD 解决了许多业务系统的架构问题

- 个人博客、社交网络、课程信息系统等相对简单的信息管理系统
- 特点：业务对象比较稳定、一次请求完成一小段局部操作

把复杂性放进 Model，不解决 Essential Complexity

- 长流程：下单 → 锁库存 → 优惠计算 → 支付 → 风控 → 发货 → 退款 → 售后 → 对账
- 强规则：修满学分 → 毕业设计 → 归还资产 → 特殊政策 → 毕业
- 会计账本：复式记账法

复式记账：同一笔业务，保持会计等式

复式记账：同一笔交易的两个方面

公司向银行借入 10,000 元

同一张记账凭证

借：银行存款	贷：短期借款
10,000 元	10,000 元
资产增加	负债增加

合计借方 = 合计贷方

资产	=	负债	+	所有者权益
+10,000		+10,000		保持不变

借 / 贷是记账方向，不等于收入 / 支出

记账架构： Event Sourcing & LLM as a Compiler

从原始凭证到可执行的分录

- 原始凭证 (事件) → 分录 (报表_支出_财务费用(CNY(75)), ...) → State Change
- 借贷平衡是可检查的约束，不代表业务真实；分录仍须核对凭证。试算平衡的局限

```
@Curry
def pay_payroll(state, category, amount, *, kind):
    """按实际付款金额结算薪酬或个人劳务；代扣款先单独记账，再支付净额。"""
    flow = PAYROLL_FLOWS[category]
    key = payable(flow, kind == "个人劳务")
    state.require(key, amount)
    cash(state, -amount, flow, balances={key: -amount})
```

从分录到报表

[redacted]-银行收支记账

- 报表_支出_财务费用(CNY(75))
- 报表_收入_投资收益_利息(CNY("22.07"))

货币资金	-52.93	财务费用	+75.00	支付其他与经营活动有关的现金	+75.00
流动资产合计	-52.93	投资收益	+22.07	经营活动产生的现金流量净额	-75.00
资产合计	-52.93	营业利润	-52.93	取得投资收益收到的现金	+22.07
未分配利润	-52.93	利润总额	-52.93	投资活动产生的现金流量净额	+22.07
所有者权益合计	-52.93	净利润	-52.93	现金净增加额	-52.93
负债和所有者权益总计	-52.93			期末现金余额	-52.93

记账为什么麻烦?

“当前状态”同时编码了过去、现在、未来

- **过去**：合同签订
- **现在**：等待打款
- **未来**：货物交接、潜在坏账……

Event Sourcing 和复杂性分解

把事实、视图、预期分开

- **过去是事实**，不可更改
- **现在是视图**，是事实经 dependency 计算得到的
- **未来是预期**，不是事实
  - 如果世界没有按预期发展，就追加新的事实来矫正

跳出问题看问题："现在"是"过去"的结果

显式建模 Data Dependency

- 选课记录 / 成绩表 → 当前已修课程集合、学分数
- 已通过课程集合 + 培养方案 → 是否满足课程准出的毕业条件

一个可查证、可溯源的计算过程

- 解决"反悔将导致多处修改，遗漏一处即崩溃"的问题
- 复杂性仍然可能带来不一致，但容易发现、容易修复

SQL 的 Billion Dollar Mistake 😄

一个设计缺陷：任意计算关系没有成为 First-Class Citizen

- SQL 没能把跨数据的**任意计算关系**作为 first-class citizen
- SQL 已有视图 / 物化视图；这里追问的是：跨数据库与应用代码的任意计算依赖，谁来统一维护?

现代框架以不同方式找补

- MVVM、React、Dataflow、Event Sourcing.....
- 把 **derived state / dependency** 显式化

从状态一致性到计算正确性

保存足以重建其余信息的 Source of Truth

- 其他状态尽可能作为 computation 的结果
- 复杂性没有消失，但从 **state consistency problem** 变成了 **computation correctness problem**

计算可以重跑、测试、审计、依赖分析

- 逻辑错误也更好补救：修正规则，从某个点开始使用新规则计算
- 复现当时的结果，需要当时的规则与外部输入：哪一年的培养方案、哪一天的汇率？Event Sourcing
- 重放可以重算余额，不能再次打款。隔离外部副作用